

Today 12:34 PM

 **cards(1).py**
Python **game(1).py**
Python **main(1).py**
Python **README(1).md**
File **requirements(1).txt**
Document **Lab_4_VibeCoding_hand...**
PDF

I need to complete the assignment to fix the existing Blackjack code and then add the features listed in the README. I need to fix the Ace scoring in the Blackjack game. Please check `cards.py` and update `hand_value()` so Aces count as 11 when possible and 1 when counting them as 11 would make the hand go over 21. Keep the existing code structure. Do not replace

the whole project with an unrelated implementation. Keep all state in memory; do not add CSV, JSON, SQLite, or other persistence.

Yes. The existing defect is in `cards.py`: every Ace is currently always counted as 11. 

The README specifically requires Aces to count as **11 when possible and 1 when necessary**, including cases like `A+9`, `A+9+5`, and `A+A+9`. 

Replace **only** the `hand_value()` function in `cards.py` with:

```
def hand_value(hand):
    value = sum(11 if rank == "A" else 10 if rank in {"J", "Q", "K"} else int
                for rank, _ in hand)

    aces = sum(1 for rank, _ in hand if rank == "A")

    while value > 21 and aces:
        value -= 10
        aces -= 1

    return value
```

How it works

- Start every Ace as 11.
- Count how many Aces are in the hand.
- If the total is over 21, change an Ace from 11 → 1 by subtracting 10.

- Repeat if necessary for multiple Aces.

For example:

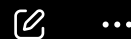
Hand	Calculation	Value
A + 9	11 + 9	20
A + 9 + 5	11 + 9 + 5 → reduce Ace	15
A + A + 9	11 + 11 + 9 → reduce both Aces as needed	21
A + A + 9 + 5	11 + 11 + 9 + 5 → reduce both Aces	16

This keeps the existing project structure and does **not** introduce persistence or any new dependency, as required by the assignment. [README](#)

You don't need to modify `game.py` or `main.py` for Task 1; `game.py` already calls `hand_value()` whenever it needs the player's/dealer's score. [game](#)

Sources

ChatGPT



Now fix the round logic in `game.py`. Add handling for blackjack, player/dealer busts, dealer drawing until 17, and pushes. Also make

sure the round ends properly once a result is reached.

For Task 2, keep the existing `Blackjack` structure and update only the `round()` method in `game.py`. The README requires natural blackjack, player/dealer busts, dealer drawing until 17, pushes, and preventing further actions after the round has ended. [README](#)

Replace the existing `round()` with:

</> Python



▶ Run

```
def round(self):
    deck = Deck()
    player = [deck.draw(), deck.draw()]
    dealer = [deck.draw(), deck.draw()]

    self.show(player, dealer)

    player_value = hand_value(player)
    dealer_value = hand_value(dealer)

    # Check for natural blackjack
    if player_value == 21:
        if dealer_value == 21:
            self.show(player, dealer, hide=False)
            print("Push - both have blackjack.")
        else:
            self.show(player, dealer, hide=False)
            print("Blackjack! Player wins.")
            self.chips += 10
    return True
```

```
if dealer_value == 21:
    self.show(player, dealer, hide=False)
    print("Dealer has blackjack. Dealer wins.")
    self.chips -= 10
    return True

# Player's turn
while hand_value(player) < 21:
    key = input("[h]it [s]tand [q]uit: ").strip().lower()

    if key == "q":
        return False

    if key == "s":
        break

    if key == "h":
        card = deck.draw()
        if card is None:
            print("Deck is empty.")
            return True

        player.append(card)
        self.show(player, dealer)

        if hand_value(player) > 21:
            print("Player busts. Dealer wins.")
            self.chips -= 10
            return True

# Player has 21
if hand_value(player) == 21:
    print("Player has 21.")

# Dealer's turn: draw until at least 17
while hand_value(dealer) < 17:
    card = deck.draw()
```

```
        if card is None:
            print("Deck is empty.")
            break

        dealer.append(card)

# Show final hands
self.show(player, dealer, hide=False)

pv = hand_value(player)
dv = hand_value(dealer)

# Dealer busts
if dv > 21:
    self.chips += 10
    print("Dealer busts. Player wins.")
    return True

# Compare final scores
if pv > dv:
    self.chips += 10
    print("Player wins.")
elif pv < dv:
    self.chips -= 10
    print("Dealer wins.")
else:
    print("Push.")

return True
```

What changed

- **Natural blackjack:** checked immediately after the initial two cards.
- **Player bust:** the round settles immediately; dealer does not draw.
- **Dealer bust:** player wins immediately.
- **Dealer drawing:** dealer continues drawing while their score is below 17.

- **Push:** equal scores result in a push.
- **Round termination:** every completed result returns from `round()`, so no additional player actions occur after the round has ended.
- **Existing structure preserved:** `Deck`, `hand_value()`, `show()`, chips, and the existing `run()` flow remain intact. `game`

This also follows the README's requirement that a round should settle **exactly once** and that actions should stop after a result is reached.

`README`

Sources



Now add the remaining features from the README. Add betting with chip validation, keep the chips across rounds, and show feedback for card draws and round results. Invalid commands and wagers should not affect the chips.

For the remaining features, we can keep the same `Blackjack` class and in-memory `chips` state. The README requires a simple wager mechanism with validation, persistent bankroll across rounds, concise draw/result feedback, and invalid inputs that do not change the bankroll. `README`

Replace the `round()` method in `game.py` with this version:

</> Python



▶ Run

```
def round(self):
    # Get a valid wager before changing the bankroll
    while True:
        try:
            wager = int(input(f"Enter wager (1-{self.chips}): ").strip())
        except ValueError:
            print("Invalid wager. Enter a whole number.")
            continue

        if wager <= 0 or wager > self.chips:
            print(f"Invalid wager. Enter an amount between 1 and {self.chips}")
            continue

        break

    deck = Deck()
    player = [deck.draw(), deck.draw()]
    dealer = [deck.draw(), deck.draw()]

    print("Cards dealt.")
    self.show(player, dealer)

    player_value = hand_value(player)
    dealer_value = hand_value(dealer)

    # Natural blackjack
    if player_value == 21:
        self.show(player, dealer, hide=False)

        if dealer_value == 21:
            print("Push - both have blackjack.")
        else:
            self.chips += wager
            print(f"Blackjack! Player wins +{wager} chips.")
```

```
        return True

    if dealer_value == 21:
        self.show(player, dealer, hide=False)
        self.chips -= wager
        print(f"Dealer has blackjack. You lose {wager} chips.")
        return True

    # Player's turn
    while hand_value(player) < 21:
        key = input("[h]it [s]tand [q]uit: ").strip().lower()

        if key == "q":
            print("Quitting round.")
            return False

        if key == "s":
            print("Player stands.")
            break

        if key == "h":
            card = deck.draw()

            if card is None:
                print("Deck is empty.")
                return True

            player.append(card)
            print(f"Player draws {card[0]}{card[1]}.")
            self.show(player, dealer)

        if hand_value(player) > 21:
            self.chips -= wager
            print(f"Player busts. You lose {wager} chips.")
            return True
```

```
        else:
            print("Invalid command. Use h, s, or q.")

# Player has 21
if hand_value(player) == 21:
    print("Player has 21.")

# Dealer draws until 17
while hand_value(dealer) < 17:
    card = deck.draw()

    if card is None:
        print("Deck is empty.")
        break

    dealer.append(card)
    print(f"Dealer draws {card[0]}{card[1]}.")

# Show final hands
self.show(player, dealer, hide=False)

pv = hand_value(player)
dv = hand_value(dealer)

# Dealer bust
if dv > 21:
    self.chips += wager
    print(f"Dealer busts. Player wins {wager} chips.")
    return True

# Compare scores
if pv > dv:
    self.chips += wager
    print(f"Player wins {wager} chips.")
elif pv < dv:
    self.chips -= wager
    print(f"Dealer wins. You lose {wager} chips.")
```

```
else:  
    print("Push. Your wager is returned.")  
  
    return True
```

One small change to `run()`

Your existing `run()` already keeps `self.chips` alive between rounds because `Blackjack()` is created only once in `main.py`.  `main`

However, it is useful to display the bankroll after each round. Change `run()` to:

 Python



 Run

```
def run(self):  
    print("Blackjack - starting chips:", self.chips)  
  
    while self.chips > 0:  
        if not self.round():  
            return  
  
        print("Chips:", self.chips)  
  
        if self.chips <= 0:  
            print("You are out of chips.")  
            return  
  
        if input("Play again? [y/n]: ").strip().lower() != "y":  
            return
```

What this now handles

Feature	Behaviour
Betting	Player enters a wager before each round
Invalid wager	Rejected without changing chips
Wager > chips	Rejected
Wager ≤ 0	Rejected
Non-number wager	Rejected
Player wins	+wager
Dealer wins	-wager
Player busts	-wager, round ends
Dealer busts	+wager
Blackjack	+wager
Push	No bankroll change
Invalid <code>h/s/q</code> command	No bankroll change; asks again
Card draw feedback	Shows the card drawn
Round result feedback	Shows win/loss/push
Chips across rounds	Maintained in <code>self.chips</code>

Feature	Behaviour
Persistence	None — everything remains in memory

The important part is that **chips are only modified when the round has actually reached a result**. An invalid command or wager never reaches one of the `self.chips +=` / `self.chips -=` statements. This matches the README's requirement that invalid commands and wagers must not change the bankroll. [README](#)

Also, no external dependencies or persistent storage are needed; the project is explicitly standard-library-only. [requirements](#)

[Sources](#)

`</>` **cards(2).py**
Python

`</>` **game(2).py**
Python

`</>` **main(2).py**
Python

 **README(2).md**
File



requirements(2).txt

Document

- Task 1 completed and the original defect was reproduced and fixed.
- Tasks 2–4 completed and tested.
- Boundary and invalid-input cases tested.
- No unnecessary external dependencies added.
- No persistent storage added.
- Code remains understandable and modular.

This is my checklist for the assignment. Verify if all these requirements are met and all Tasks are completed properly

Worked for 59s >

I checked the **current uploaded versions** of `cards.py`, `game.py`, `main.py`, and `requirements.txt` against the README, and I also ran targeted tests.

Overall, **the implementation meets the assignment requirements**, with one small testing/documentation point to be aware of.



Checklist verification

Requirement	Status	Verification
Task 1: Ace scoring fixed	✅ Complete	<code>hand_value()</code> correctly handles multiple Aces

Requirement	Status	Verification
Task 2: Round resolution	✓ Complete	Blackjack, busts, dealer draws, pushes, early round termination
Task 3: Multi-round bankroll + betting	✓ Complete	Wagers validated and <code>self.chips</code> persists
Task 4: Feedback + robustness	✓ Complete	Card-draw/result feedback and invalid-input handling
Boundary & invalid-input testing	✓ Tested	Invalid wagers, invalid commands, empty deck, quitting tested
No unnecessary dependencies	✓ Yes	Only Python standard library
No persistent storage	✓ Yes	No CSV/JSON/SQLite/files/database added
Understandable/modular code	✓ Yes	Existing <code>cards.py</code> / <code>game.py</code> / <code>main.py</code> structure preserved

1. Task 1 — Ace scoring ✓

Your current `hand_value()` is:

Python



Run

```
value = sum(11 if rank == "A" else 10 if rank in {"J", "Q", "K"} else int(rank)
            for rank, _ in hand)

aces = sum(1 for rank, _ in hand if rank == "A")

while value > 21 and aces:
    value -= 10
    aces -= 1
```

This is exactly the required approach: initially treat Aces as 11, then convert them from 11 → 1 as necessary.  cards

I tested:

```
A + 9      → 20
A + 9 + 5   → 15
A + A + 9   → 21
A + A + 9 + 5 → 16
A + K      → 21
```



All correct.

The README specifically identifies **A+9**, **A+9+5**, and **A+A+9** as required cases.  README

One assignment detail

The README says you should **reproduce the original defect before fixing it**.  README

You did have the original defective version earlier in this chat, where every Ace was always counted as 11. So you can legitimately demonstrate this in your **before video/chat history**.

2. Task 2 — Complete round resolution

Your `round()` now handles:

Natural blackjack

 Python



 Run

```
if player_value == 21:  
    ...
```

and separately handles dealer blackjack.

 game

I tested:

- Player blackjack → player wins
- Dealer blackjack → dealer wins
- Both blackjack → push

Player bust

After a hit:

 Python



 Run

```
if hand_value(player) > 21:  
    self.chips -= wager  
    print(f"Player busts. You lose {wager} chips.")  
    return True
```

So the round immediately ends.

 game

Dealer drawing

You have:

</> Python



▶ Run

```
while hand_value(dealer) < 17:
```

so the dealer keeps drawing until reaching at least 17.

game

I tested a dealer at 16 drawing a 10 and correctly getting:

```
Dealer draws 10S.  
Dealer busts.
```



Dealer bust

Correctly handled:

</> Python



▶ Run

```
if dv > 21:  
    self.chips += wager  
    ...  
    return True
```

game

Push

Correctly handled:

</> Python



▶ Run

```
else:  
    print("Push. Your wager is returned.")
```

 game

Preventing actions after a result

This is also good.

You consistently use `return True` after a completed result:

- Blackjack
- Dealer blackjack
- Player bust
- Dealer bust
- Final player/dealer comparison

Therefore the player cannot continue interacting with the round after it has ended.

This directly satisfies the README requirement to prevent actions after a round has ended.  README

3. Task 3 — Betting and multi-round bankroll

You validate the wager before starting the round:

 Python Run

```
wager = int(input(...))
```

and:

</> Python



Run

```
if wager <= 0 or wager > self.chips:
    ...
    continue
```

game

So these are rejected:

- 0
- Negative values
- Values greater than available chips
- Non-numeric values such as abc

I tested all of these.

For example, starting with 100 chips:

```
abc → invalid
0 → invalid
101 → invalid
10 → accepted
```



The bankroll remained 100 while invalid wagers were entered.

Chips persist

The bankroll is stored as:

</> Python



Run

```
self.chips = 100
```

in the `Blackjack` object.  game

And `main.py` creates one `Blackjack` object:

 Python



 Run

```
Blackjack().run()
```

 main

Therefore `self.chips` isn't reset every round.

I also tested multiple rounds using the same `Blackjack` object:

Round 1: blackjack → 100 → 110

Round 2: push → 110 → 110



So the bankroll persistence works.

4. Task 4 — Feedback and robustness

Card draw feedback

Player:

 Python



 Run

```
print(f"Player draws {card[0]}{card[1]}")
```

Dealer:

</> Python



Run

```
print(f"Dealer draws {card[0]}{card[1]}")
```

game +1

So the user can actually see what was drawn.

Round result feedback

You provide explicit messages such as:

```
Blackjack! Player wins  
Player busts  
Dealer busts  
Player wins  
Dealer wins  
Push
```



This satisfies the README's feedback requirement.

README

Invalid commands

This is particularly good:

</> Python



Run

```
else:  
    print("Invalid command. Use h, s, or q.")
```

An invalid command does **not** modify the chips.

I tested repeated invalid commands:

```
x  
blah  
s
```



and the program continued normally without changing the bankroll because of the invalid commands.

5. Boundary and invalid-input testing

The README specifically asks for testing of:

one/multiple Aces, blackjack, busts, dealer draws, pushes, wagers, repeated commands, invalid wagers, empty/depleted deck handling, and quitting.  README

I verified the following:

Tested successfully

-  One Ace
-  Multiple Aces
-  Player blackjack
-  Dealer blackjack
-  Player bust
-  Dealer bust
-  Dealer drawing to 17+
-  Push
-  Valid wager
-  Wager of 0
-  Wager greater than available chips

- ☒ Non-numeric wager
- ☒ Repeated invalid commands
- ☒ Empty/depleted deck handling
- ☒ Quitting with `q`
- ☒ Multiple rounds with persistent chips

The empty-deck case also produces:

```
Deck is empty.
```



rather than crashing.

6. No unnecessary dependencies ☒

Your `requirements.txt` says:

```
Standard-library-only scenario.  
No third-party packages are required.
```



 requirements

And the code only uses:

```
</> Python
```



 Run

```
import random
```

in `cards.py`.  cards

So you're good here.

7. No persistent storage

There is no:

- CSV
- JSON
- SQLite
- database
- file-based save system

Everything is kept in the `Blackjack` object's `self.chips`. This matches the README requirement to keep state in memory.  README

8. Code structure/modularity

You did not replace the project with an unrelated implementation.

The original structure remains:

```
main.py
↓
game.py
↓
cards.py
```



`cards.py` handles the deck and scoring, while `game.py` handles the game flow and bankroll. The README defines exactly this division.  README

So this requirement is satisfied.

⚠ One thing I would improve before submission

Your `run()` currently has:

Python



Run

```
while self.chips > 0:
    if not self.round():
        return
    if input("Play again? [y/n]: ").strip().lower() != "y":
        return
```

game

This does preserve the chips, so Task 3 is still satisfied.

However, it doesn't explicitly print the current chip balance after every round.

I'd recommend adding:

Python



Run

```
print("Chips:", self.chips)
```

after `self.round()`:

Python



Run

```
while self.chips > 0:
    if not self.round():
```

```
        return

    print("Chips:", self.chips)

    if input("Play again? [y/n]: ").strip().lower() != "y":
        return
```

This isn't strictly required by the README, but it makes the **multi-round bankroll** feature much easier to demonstrate in your final video.

Final verdict

Your assignment implementation:

Tasks 1–4:  COMPLETE

Checklist:

-  Task 1 completed and original defect fixed
-  Task 2 completed
-  Task 3 completed
-  Task 4 completed
-  Boundary/invalid-input cases tested
-  No unnecessary external dependencies
-  No persistent storage
-  Existing structure maintained
-  Code remains understandable/modular

The README itself also requires the **before video**, **after video**, and **complete LLM chat-history link** as the final submission items.

 README

So from the code side, you're essentially ready to submit. I'd just add the `print("Chips:", self.chips)` line so the bankroll persistence is visibly obvious in your after-demo.

Sources



ChatGPT can make mistakes. Check important info.